When Can an Agent Act Without Thinking?

Execution contracts and stop-record diagnostics for an LLM agent on ARC-AGI-3

Yuto Kojima

ARC Prize 2026 Paper Track · linked ARC-AGI-3 submissions: 56836939 (36.14) and 57877794 (30.68)

Abstract. We study how an LLM agent can retain a successful trajectory as a procedure that can be executed without repeated reasoning. We add ECA (Existential-Cognition Architecture), a middleware layer, to a Kaggle harness for ARC-AGI-3. An execution contract governs when a routine may start, how its predictions are checked, and when execution must stop. The contract covers two routine sources: replay of successful trajectories and parameterised models of game mechanics. Both act without the LLM while the observed outcomes match their predictions, and return control after a mismatch. On eight public games, all 182 level replays matched the stored observations without generating LLM tokens. Levels completed per run increased from a control mean of 34.5 to 46 with the second library version. All 16 planted divergences stopped at the specified action; the LLM completed 11 of 16 levels from the resulting state, compared with 7 of 16 from the start. Offering the same certified plans as advice completed the same target levels in the registered comparison, but required more LLM turns and generated tokens than direct execution; in an additional authority run, a level that could not be certified fell back to the LLM, and that run's assigned levels then used more generated tokens than either advice run. Existing routines completed no levels on 33 unseen games. With LLM-written transition models, each of 13 gate stops, in five level attempts, was followed by a new version of the model; in three attempts a later version agreed with every recorded transition and had a plan, and on two levels the harness then completed the level, in about as many actions as the LLM alone. Consistency with recorded transitions did not ensure correct predictions on unobserved transitions. We report the costs and reproducibility of reuse on known tasks, together with the observed limits on unseen tasks.

1 Introduction: retaining solutions as reusable abilities

What allows an agent to execute a task reliably and at low cost after it has solved that task once? This paper studies a mechanism for retaining successful LLM-agent behaviour as executable procedures and returning to reasoning when those procedures no longer apply.

Our control agent has no path for retaining a new success from one run as a procedure that can execute directly in the next. Two runs of one configuration completed 31 and 38 levels across eight public games. Across 83 collected runs, 45 distinct levels were completed at least once, but the mean was 33.3 per run. With the built-in solver disabled (Section 3.3), the LLM spent 0.36–0.51 M generated tokens per run on the 13 levels assigned to the solver, and still failed some of them. Access to a record of earlier actions does not, by itself, provide a procedure for reproducing them.

Saving a successful procedure can reduce the cost of solving the same task again. Its applicability must nevertheless be checked. Identical frames can correspond to different internal states, and an execution may produce an outcome that was not predicted. Reuse therefore requires an initial applicability test, monitoring during execution, and a stopping rule.

ECA performs these checks between the LLM and the environment. The name Existential-Cognition Architecture comes from the original concept behind it: missing or inconsistent internal representations of self, world and value are treated as the agent's own occasions for learning and self-correction. This paper evaluates one part of that concept, the verification, reuse and execution control of routines, in which a routine's authority to act rests only on transitions actually observed and verified in the environment, the evidence E of Section 3. It applies one execution contract to trajectory replay and to models of particular

game rules, called mechanic families. A routine receives authority when its applicability conditions hold. Each action's outcome is checked against its prediction. At the first mismatch, authority is revoked and control returns to the LLM. This contract prevents further automatic execution by that routine after an observed mismatch; it does not prevent the mismatching action itself.

The division of labour is informed by accounts of automatic and supervised skilled behaviour. Riding a bicycle illustrates the distinction between a practised movement and a response to an unexpected situation. Relevant concepts include instance-based automaticity (Logan, 1988), motor schemas (Schmidt, 1975), supervisory control in novel or erroneous situations (Norman & Shallice, 1986), forward models of movement outcomes (Wolpert et al., 1995), and reliability-based arbitration between habitual and deliberative control (Daw et al., 2005). These are design analogies, not a claim to reproduce human motor learning. The work began as middleware for physical AI; here we evaluate its execution control in discrete game environments.

We distinguish two evaluation settings. On known public games, we measure retention and reuse of successful procedures, execution cost, and continuation after a deviation. On unseen games, we measure the applicability of existing routines. The latter measurements include separate game collections rather than only Kaggle's hidden evaluation (Sections 2 and 5.6). The contributions are:

1. **A shared execution contract for two routine sources (Section 3).** Trajectory replay is matched by frame hashes; mechanic families are checked against observed transitions. Routine-specific applicability checks are combined with post-action verification, revocation, and return of control to the LLM. Successful trajectories are harvested between runs and added to the library after replay verification.
2. **Measurements under a competition model and execution budget (Section 5).** We evaluate repeated inference, advice versus direct execution, replay consistency, continuation after deviation, applicability to unseen games, and LLM-written models with repair after a stop. The linked Kaggle entries are ECAOD v1 (36.14) and the failure-note variant (30.68). The additional library experiments are reported separately from these entries.
3. **Failure analysis using stop records (Section 6).** Predicted and observed outcomes are recorded together, allowing candidate causes and the measurements needed to distinguish them to be specified.

LLM-free replay and fallback after deviation have precedents in GUI agents (Section 4). We do not claim novelty for the cognitive analogy. The focus is the execution contract and measurements of its cost and scope.

The mechanic families were pre-authored from the public games before evaluation and served as the solver of the submitted kernels. This paper does not propose a method for acquiring them and does not claim that they generalise; Section 5.6 reports no completions on unseen games. Here they are a fixed routine source with which the contract itself can be measured: authority against advice for the same plans, revocation at a mismatch, and hand-back. Section 5.7 asks what changes when the LLM writes the model instead, including repair after a stop.

2 Setting

ARC-AGI-3. The benchmark (ARC Prize Foundation, 2026) consists of turn-based interactive games rendered as 64×64 frames with 16 colours. Actions include keyboard-like inputs, clicks, and RESET. Each game contains multiple levels, scored against human action baselines. In hidden evaluation, entrants play each previously unseen game once. Our Kaggle entries run offline on one GPU within the competition time limit.

Harness and model. We use Daniel Franzen's public Milestone 2 notebook (Franzen, 2026). It runs Tufa Labs' Duck harness (Bessis et al., 2026; `tufa-arc-agi-framework`) and serves Qwen3.8 Flash-Next through vLLM with W4A16 quantisation and a speculative drafter. The LLM receives the frame and history and acts by generating Python that calls `action([...])`. Long prompts are replaced by summaries. ECA sits between the harness and the game, with individual features enabled or disabled through environment variables.

Local evaluation. Online comparisons use eight public games (59 levels), a Colab G4 instance, the same model, and a budget of 60 minutes per game. Unseen-game measurements use 25 ARC-AGI-3-format games published by NVIDIA (Apache-2.0, NV25) and eight games by another author (arcint). These are measurement sets; no adjustment was made from their results. Offline analyses of recorded trajectories are identified in the relevant sections and Appendix A.

3 The execution contract

A reusable procedure needs conditions for starting, continuing, and stopping before control returns to the LLM. Table 1 relates these decisions to the terms used below.

Decision	Term
Does the model explain the recorded transitions?	Closure on observed evidence
Does the routine meet its conditions in the present state?	Certificate
How far may it execute without an LLM call?	Lease (execution authority)
Has an observed outcome differed from its prediction?	Revocation and return of control

Table 1. Decisions made by the execution contract.

3.1 Closure, certificate, and lease

We treat a level as a deterministic transition system with hidden state s. The agent observes a frame f = render(s). A routine ρ has a predictor $P_\rho(h, a)$ for the next frame, given the level history h and action a. Evidence E is the set of recorded transitions (h_i, a_i, f_{i+1}).

- A routine is **closed** on E when it predicts all recorded next frames. Families compare frames excluding a four-cell border containing the HUD. Closure here denotes agreement with a finite evidence set.
- A routine receives a **certificate** when it agrees with the evidence and has a plan that completes the level under its model. For replay, the stored successful trajectory is the evidence, and the start frame must match the stored start frame.
- A certified routine receives a bounded **lease**. Observations are checked after every action, and the first mismatch revokes the lease. Control returns to the LLM. What is checked is defined by each source: replay compares each observed frame with the stored one; a family checks the prediction its own verifier defines, which is a hash of the whole predicted next frame for some families and, for others, the parsed entity positions, a static board signature and the remaining plan (Appendix B); a family without an exact per-action prediction is re-certified on the new frame. The contract is shared; the verification target is not. Where implemented, notes report the stop position and mismatch (Table 2).

As in runtime assurance, an external mechanism monitors execution conditions. The fallback here is an LLM, not a verified safety controller. Sections 5–6 examine the following three properties.

P1: replay consistency can be checked. In a deterministic environment, the same initial state and action sequence produce the same observations. A matching start frame alone does not establish equality of the internal state. Replay continues only while observations match the stored sequence and stops at the first difference. Differences that remain unobservable cannot be detected.

P2: execution stops at the first visible counterexample. Per-action checks stop a routine at its first observed prediction mismatch. The action causing the mismatch has already occurred; the contract does not undo its effects.

P3: a frame may omit distinctions needed for prediction. If the same frame and action are observed to lead to different next frames, no deterministic predictor using only that frame and action can agree with all those observations. Adding history can distinguish some cases. This is related to the apartness approach to active automata learning (Vaandrager et al., 2022).

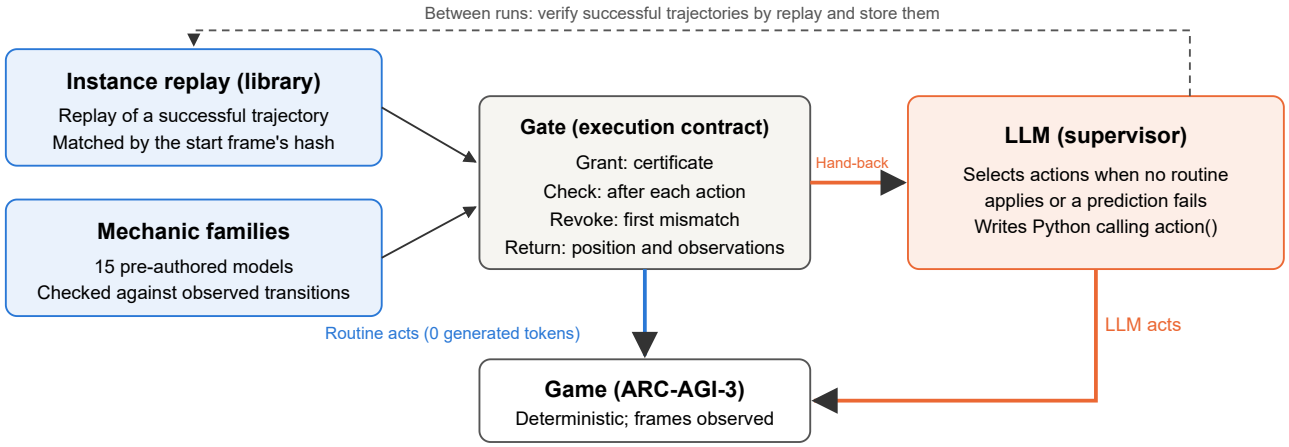


Figure 1. The shared execution contract. Each routine source has applicability checks and post-action verification. A mismatch stops the routine and returns control to the LLM. Successful trajectories enter the library between runs. ECA denotes the whole middleware, including the gate and the components in Section 3.2, not only the mechanic families.

3.2 ECA components

ECA analyses frame components and action effects and decides when a routine may execute for the LLM. This paper evaluates the contract for execution, stopping, and reuse rather than ECA as a whole. Table 2 identifies the components and configurations in which they were used. Section 5 evaluates systems under the specified conditions; it does not isolate every component's individual effect.

Component	Operation	Contract role	Analogy	Used in
Family perception	Each family extracts its own actors, walls, axes, targets, or pieces. A shared perception representation is not required.	Constructs predictive state	Perceptual parsing	S1, S2, L
Families and certificates	Fifteen parameterised models use structure-based applicability checks and retain the reason a check passes or fails.	Checks agreement with evidence	Motor schemas (Schmidt, 1975)	S1, S2, L
Gate	Detection, screening, arbitration, projection, and verification. It selects at most one family per frame and releases a bounded plan prefix. Each step must preserve the certificate and advance the plan.	Grants, maintains, and revokes authority	Supervisory control (Norman & Shallice, 1986)	S1, S2, L
Outcome ledger	Records completion and revocation per family and episode. With supporting outcomes, the lease grows from one action to two, four, and the whole plan.	Sets execution scope	Reliability-based arbitration (Daw et al., 2005)	S1, S2, L
Episode mechanic memory	Stores coordinate-independent schemas marked observed, supported, or verified. Verified schemas can authorise a rebound planner on a later level; supported schemas provide bounded advice.	Retains evidence across levels	Schema transfer	S1, S2, L

Component	Operation	Contract role	Analogy	Used in
Host note	Reports the routine's actions and the location of the level's starting frame.	Handover	Report to the supervisor	S2, L
Failure note	Reports the family model, the mismatching action, and the difference between prediction and observation (Appendix B).	Communicates the stop	Error signal	S2
Scheduler	Scans at a level's start and after 20 actions without a level change. It does not re-evaluate the same frame. Always-on scanning had cost 1.5–2 s per scan, 1.3–2.7 scans per action.	Limits checking cost	—	S1, S2, L
Instance library	Stores start and post-action frame hashes and replays successful trajectories (Section 3.3).	Second routine source	Instance retrieval (Logan, 1988)	L (additional experiments)

Table 2. Components used in this study. S1: submission 56836939 (ECAOD v1); S2: submission 57877794 (failure note); L: local experiments. Cognitive correspondences are design analogies.

Checks with insufficient support do not authorise execution. Lease length is adjusted using execution records. Section 5 reports results where routines apply; Sections 5.6–5.7 and 6 describe their limits and prediction mismatches.

3.3 Two routine sources

Mechanic families. Fifteen parameterised models were authored before evaluation from the public games. They include stride_maze, sand_filter, program_execution, linked_extent, rule_rewrite, reflection_orbit, anchor_overlap, region_paint, shape_marker, hierarchical_slots, cycle_permutation, control_marker, and mirror. Parameters are fitted to observed transitions, after which applicability and predictions are checked. Most families plan using bounded breadth-first search. A prefix executor executes the authorised portion one action at a time. In this paper the families are the measured routine source, not a proposed way of obtaining models (Section 1).

In the submitted kernel, the toolbox runs at a level's start and after 20 actions without a level change, without re-evaluating the same frame. Later configurations report executed actions through a host note. The families, planner, and executor together are the built-in solver added in this work, not a component of Franzen's original notebook. Its assigned public levels are ar25 L1–3, cn04 L1–2, re86 L1, s5i5 L1–2, and tn36 L1–5: 13 levels used in Section 5.2. Appendix B gives a concrete example.

Instance replay. These additional experiments use the agent's successful histories and are not part of either linked Kaggle entry. Histories are replayed on a local engine and split by level. Among the collected candidates, the shortest procedure completing a level from its canonical start is retained and shortened under replay verification. The start-frame hash and the hash after every action are stored. At runtime, replay continues only while these hashes match. This is a verified cache in a deterministic environment; the evaluation concerns the effect of retaining known-task solutions. The source trajectories come from the whole agent, including both LLM and solver actions. Of the 46 stored levels, 45 had been completed by both at some point, while s5i5 L7 was added from an LLM-only success.

3.4 Kaggle submissions

ECAOD v1 (submission 56836939). We added an on-demand family toolbox to Franzen's Milestone 2 notebook. Repeated scores from the same kernel were 36.14, 32.60, and 23.67 (mean 30.8). Another run returned 0.00, with no log available to entrants; including it gives a mean of 23.1. The original notebook scored 29.69 and 27.58 (mean 28.6). An always-on toolbox version scored 20.94 and scanned 1.3–2.7 times per action, at 1.5–2 s per scan, even on games with no certificate. With few repetitions and a range of about 12 points even among the nonzero same-kernel scores, we do not claim a hidden-set improvement.

Failure-note variant (submission 57877794, 30.68). This adds mismatch reporting to a later configuration whose scores were 29.58 and 28.10. When a family fails after at least three verified actions, the LLM receives the model, the action, and the difference between prediction and observation. Neither submission includes the instance library. No replay entry is available in advance for a particular hidden game.

Development history. Submission 54666986 in July 2026 used an earlier harness and Qwen3.6-27B. One family executed certified actions under a lease without adding advisory text to the prompt. It scored 1.86 and led the public leaderboard at the time. Contemporary variants scored 0.75 with advice, 0.42 with 12 families and broader advisory text, and 0.90 with the LLM alone. Repeating the 1.86 kernel with the same deterministic configuration returned 1.10, so the higher score is treated as a single high observation. A subsequent local comparison produced no clears with advice, while execution under a four-action lease cleared levels in all six cells. In tu93, advice increased thinking tokens per action 11.5-fold. Section 5.2 repeats the comparison with the current harness.

4 Related work

Switching between fast and slow processing. SwiftSage (Lin et al., 2023), SOFAI (Booch et al., 2021), AdaPlanner (Sun et al., 2023), and WebATLAS (Cheng et al., 2025) use different processing paths or feedback to control agent behaviour. Combining a fast path with deliberation is not our novelty claim. We focus on the applicability conditions of LLM-free routines and their termination after prediction mismatches.

Retaining and replaying successful trajectories. SkillDroid (Chen et al., 2026) compiles successful Android trajectories into skills, replays them without the LLM, checks UI state at each stage, and returns control with the completed stages. It uses regexes, embeddings, and LLM confirmation for matching and recompiles frequently failing skills. MobiMem (Liu et al., 2025), motivated by procedural memory, reuses action records with embedding-based applicability and fuzzy element matching. We use exact frame-hash matching in deterministic games, place model-based families and replay under a common execution contract, and measure costs and applicability under the competition model.

Skill libraries without weight updates. Voyager (Wang et al., 2024) stores executable code skills that the LLM retrieves and combines. Section 5.2 compares presenting a plan to an LLM with delegating its execution directly. It is not a cost comparison with Voyager itself.

World models for ARC-AGI-3. Schema (Zeng et al., 2026) executes a committed action sequence with per-step checks against a world model and returns to deliberation after a mismatch. Rodionov (2026) stops at the first disagreement between an executable model and observation. Twin (Skoutnev et al., 2026) checks model fit to recorded transitions before execution, and Tycho (Lehmann et al., 2026) treats games as rendered deterministic Moore machines. Comparisons with this work need to distinguish the competition model, execution budget, and cross-run instance reuse evaluated here.

Runtime assurance and consolidation. Simplex (Sha, 2001) switches from an advanced controller to a verified safety controller when execution leaves its admissible envelope. Our fallback is an LLM, so the guarantee differs. Other routes to retaining acquired knowledge include test-time training on static ARC (Akyürek et al., 2025), SOAR (Pourcel et al., 2025), and Soar chunking, which compiles reasoning into rules (Laird et al., 1986).

5 Results

Table 3 lists the evaluation questions. Appendix A identifies registrations for the comparisons and analyses of recorded trajectories. Unless stated otherwise, results concern the eight public games.

Evaluation	Section
Repeated-inference cost and run-to-run variation	5.1
Advice versus direct execution of the same plan	5.2
Registering a new success as a routine	5.3
Replay consistency and run outcomes	5.4
Continuation from a state reached before deviation	5.5
Applicability to unseen games	5.6, 6
LLM-written transition models	5.7

Table 3. Evaluation questions and corresponding sections.

5.1 Inference cost and run-to-run variation

We measured action intervals and tokens from execution logs. A replayed action took a median of 0.24 s (p90 1.09 s; n = 5,666), including the hash check. The first action of an LLM turn occurred a median of 46 s after the previous action (p90 227 s; n = 1,232); the turn generated a median of 3,300 tokens. These are action-interval measurements, not a task-level speed ratio. A run generated approximately 1.8 M tokens.

Four runs of one configuration (a1–a4) completed 37, 37, 33, and 36 levels. Forty-two levels were completed in at least one run, and 30 in all four. The resulting 12 levels with differing outcomes met the registered prediction of at least six.

Each request contained an average of 74.9 k prompt tokens. Across 32 control-family game-runs (30.9 game-hours), the prompt was replaced by a summary 144 times. Its length immediately before replacement had a median of 116 k tokens; the median interval was 20 requests, or 4.66 replacements per game-hour. Earlier experience is then passed through the summary. With the solver disabled, the LLM used 0.36–0.51 M tokens per run on the 13 assigned levels, completing 23 and 18 of 26 level-runs in the respective comparisons. Neither configuration progressed beyond tn36 L5, whereas the control with the solver reached tn36 L6 and L7 in its two runs, so the solver's advantage in levels was concentrated on tn36. The aggregate differences (32.5 and 30.5 against the control's 34.5 levels per run) lie within the spread of repeated runs, and the solver's contribution to the hidden-set score was not identified.

5.2 Advice versus direct execution of certified plans

We compared direct execution by the solver with presenting the same certified plans to the LLM as advice.

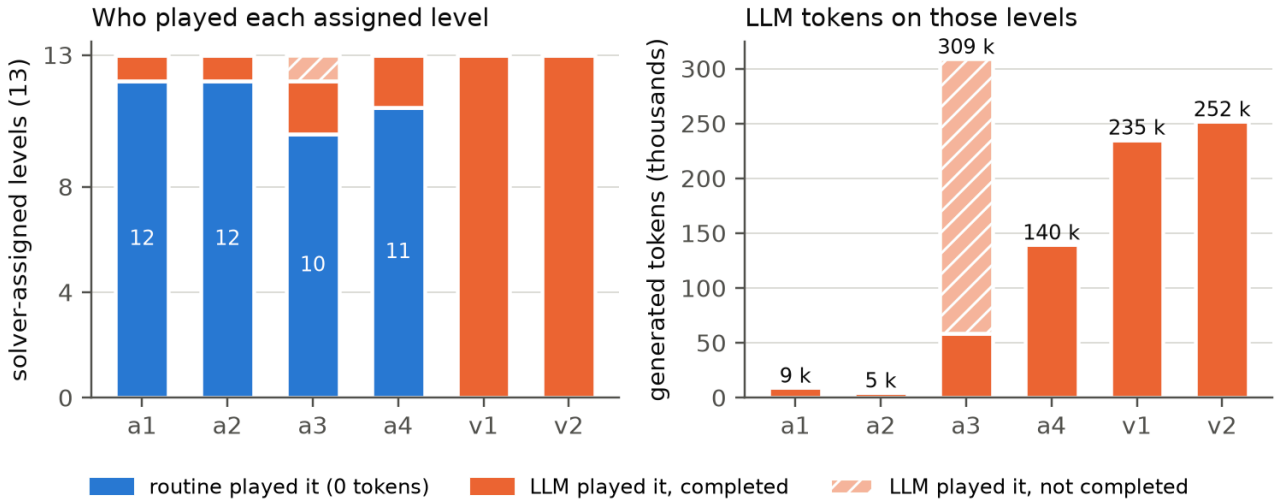


Figure 2. The 13 solver-assigned levels in four authority runs (a1–a4) and two advice runs (v1, v2). Left: whether the routine played a level (no action carried LLM tokens) or the LLM did. Right: generated tokens on the levels the LLM played; levels the routine played contribute none. The hatched part is a3's s5i5 L2, which the LLM did not complete; its tokens run to the end of the run. The registered comparison is a1/a2 versus v1/v2; a3/a4 are additional authority runs. Tokens are attributed to executed actions and are a lower bound.

In the advice condition, the harness presented the plan's next action in the LLM's next prompt instead of executing it. The message stated that the harness would not act and that following the plan was the LLM's choice. An offer was counted as followed when the next executed action matched the plan. Follow rates were 32/44 (0.73) in v1 and 59/69 (0.86) in v2.

All 13 assigned levels were completed in every run of the registered comparison, a1/a2 versus v1/v2. Completion counts were therefore equivalent under the registered decision rule. Across all four authority runs, 45 of 52 assigned level-runs were handled without an LLM turn. The other seven were handled by the LLM because certification was not obtained in time; six were completed. On s5i5 L2, a3 used 20 turns without completing the level, while a4 completed it in 17 turns.

Up to the last completed assigned level of each game, the four authority runs used 4–34 LLM turns, 5–140 k generated tokens, and 10–39 game-minutes per run. These ranges leave out a3's s5i5 L2, which was not completed; counting its tokens to the end of the run, a3's LLM-played assigned levels used 309 k tokens, more than either advice run (Figure 2). Under advice, every assigned level required LLM turns: 76 or 85 turns, 235–252 k tokens, and 71–90 game-minutes per run. Total level counts were 37, 37, 33, and 36 under authority, and 35 and 31 under advice. The assigned-level costs differed, but these repetitions do not establish an effect on total levels completed.

The advice was a roughly 45-token line specifying the next action, with the family replanning after each step. The observed cost difference corresponds primarily to how often action selection passes through the LLM, rather than to a large increase in prompt length. The model followed many offers but still incurred inference cost. The low compliance observed in the July comparison is therefore not sufficient to explain this result.

5.3 Registering a new success as a routine

In run l2, after s5i5 L1–L6 were replayed, the LLM completed L7, which was not yet in the library. It used 65 actions, 19 LLM turns, 224,901 generated tokens, and 46.5 minutes; the human action baseline was 86. The harvest extracted L7 from the successful history and shortened it to 61 actions under replay verification. In l3 and l4, L1–L7 were replayed in 226 actions with no generated tokens, and all frame hashes matched. This provides one trace from a new success to subsequent LLM-free execution.

5.4 Replay consistency and run outcomes

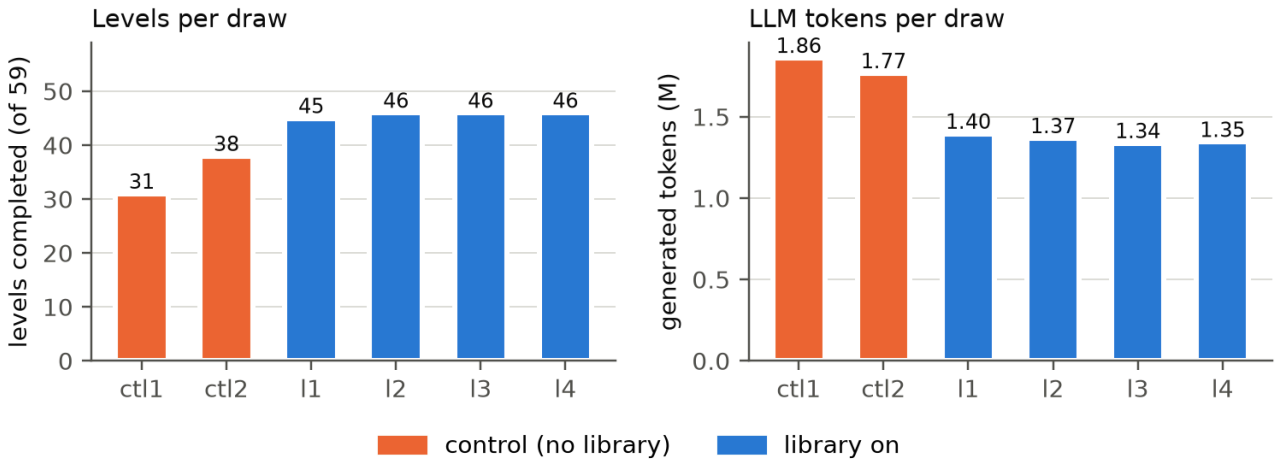


Figure 3. Levels completed and generated tokens per run. The control runs are ctl1/ctl2; l1/l2 use library v1 with 45 levels, and l3/l4 use v2 with 46. Token counts include all generated tokens recorded by the harness.

All 182 level replays matched the stored observations: 45 levels in each of two runs, followed by 46 in each of two runs. No replay stopped, and replay generated no LLM tokens. Completed levels were 31 and 38 in the controls (mean 34.5), compared with 45, 46, 46, and 46 with the library. Mean game scores changed from 42.8 to 66.5 and 67.7, while generated tokens changed from 1.82 M to 1.38 M and 1.34 M.

The library configuration also included a host-note level fix and a warm-up RESET fix. The aggregate results are therefore comparisons between system configurations. Except for s5i5 L7 in l2, completed levels in l1–l4 came from library replay, whose consistency was checked directly. Since every other completed level was a replay checked frame by frame, the two fixes could account for at most one completed level beyond replay. The token difference includes both the effect of three games ending after replay without LLM turns and any effect of the harness fixes.

Neither l3 nor l4 added a new level, and both replayed all 46 stored levels. No addition in these two runs does not establish a fixed point of future learning. The result is that the accumulated successful procedures were reproduced in the evaluated repetitions.

5.5 Continuation after a deviation

We deliberately replaced an action after approximately half the stored sequence on the last library level of each game. In condition R, replay stopped at the first mismatch and the LLM continued from the state reached. In S, the level's library entry was removed and the LLM solved it from the start. Each condition comprised eight levels in two runs.

All 16 replays stopped at the specified deviation. R completed 11/16 levels and S completed 7/16, meeting the registered criterion $R - S \geq 3$. Among completed levels, the median cost from handover or level start was 41 model actions, 112 k tokens, and 28 minutes in R, versus 117 actions, 157 k tokens, and 35 minutes in S. When the substituted action left the frame unchanged, all four levels were completed; when it moved the frame off the stored path, seven of twelve were completed.

R benefits from the progress made before handover. This tests partial reuse combined with immediate stopping, not the effect of the note alone. In other experiments, only one of five repair requests was taken up (Table 4). The R/S difference was not statistically significant in this sample (two-sided Fisher's exact test, $p = 0.29$).

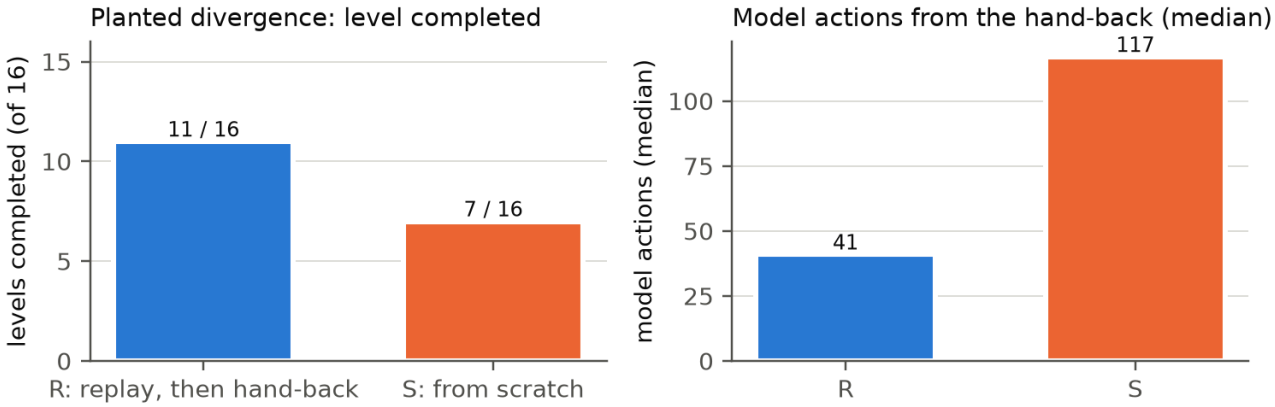


Figure 4. Continuation after deviation, with eight levels in two runs per condition. R stops replay at a mismatch and continues with the LLM from the reached state; S uses the LLM from the level's start. Action medians include only levels completed in each condition.

5.6 Applicability to unseen games

On NV25, the mirror family was certified on two of 25 games. Both certificates were revoked after the first action, without completing a level. No family was certified on the eight arcint games. In a comparison using a simple inspect-and-cycle actor instead of the LLM, the families added ten completed levels over blind action on the public games and none on NV25. The instance library contained no entries for unseen games. Existing routines therefore produced no completed levels on these 33 games. Contributions of individual components to Kaggle's hidden scores were not isolated.

We also measured opportunities to replay a prefix after RESET or game over. Repeated prefixes accounted for 0.42% of model actions on public games and 0.31% on NV25. This measures prefix-replay opportunities, not the frequency of all state revisits. In the library experiments, 20 game-runs spent 19.1 game-hours after replay and completed one new level, s5i5 L7.

5.7 LLM-written transition models

In the registered modeler condition, the LLM generated Python transition functions, `step`, and goal tests, `goal`. The harness checked models against recorded transitions, searched for plans, and executed them with per-action checks. Across two runs, a `step` function was retained on 90% of the counted levels; among levels with a `step`, 59% were near-exact on the records. The harness found plans on eight of sixteen levels with an exact model and goal and completed five levels with 198 actions of its own. Total completion nevertheless fell from 69 control levels to 55 modeler levels across two runs per condition.

Of 91 searches, 49 found no path to the goal under models consistent with every recorded transition. Thirteen of twenty executed plans stopped on the first action into an unobserved state. In a separate condition offering verification and planning as optional tools, they were used in only 2–3% of requests.

Agreement with observed transitions did not establish predictive accuracy on unobserved transitions. Limited model coverage was a major constraint in these runs, but its contribution was not isolated from verification and search costs or from allocation of the action budget. Studies using stronger models to synthesise and validate world models report high public-set performance (Zeng et al., 2026; Skoutnev et al., 2026). Two reports under other models and harnesses describe a similar pattern, not a replication of this setting. In Tycho's matched comparison (Lehmann et al., 2026), automatic repair after verification failures gave the closest match to observed transitions but a lower score (83.07 RHAE) than actor-requested delegation (88.49); the authors conclude that accurate transition prediction does not by itself ensure strong play. A post-competition report by a Kaggle participant placed 7th on the public leaderboard (Sestak et al., 2026) describes three attempts at LLM-written world models that fit the recorded transitions without clearing more levels. Whether such a change would improve this system requires a separate evaluation.

Repair after a stop. Because the gate stops a plan at its first mismatch, each stop leaves a counterexample: the action, the predicted frame and the observed frame. In the modeler condition the next tool result reported it, the LLM could rewrite `step`, and the harness re-verified, searched and executed again. We followed every stop forward in the two recorded runs (registered re-analysis; no new run). The 13 stops fell in five level attempts and are not independent. Each harness check was tied to the version of `step` in effect when it ran, and a stop was followed only through versions defined after it; a first count that dated model definitions by the request carrying them could count the stopping definition as a rewrite and is superseded (Appendix A). Every stop was followed by a new version of `step`. Nine stops were followed by a new version that the harness accepted with a plan under its near-exact rule (at most 10% of judged transitions mismatched), six of them by a version that agreed with every recorded transition; by level attempt, three of five reached such a version. On two levels the chain ended in a harness completion: on dc22 L1 three plans stopped (after 4, 1 and 11 actions; 48, 4 and 4 differing cells), each was followed by a new version that agreed with all 26, 27 and 38 recorded transitions, and the fourth plan of two actions completed the level; dc22 L2 followed the same pattern with two stops. Both levels are in one run and outside the pre-authored families. They took 40 and 72 actions, against 38/37 and 82/72 for the LLM-only controls and human baselines of 59 and 102, so the loop reproduced what the LLM alone achieves rather than exceeding it. On ls20 L3 (five stops) new versions were accepted under the near-exact rule but never agreed with every recorded transition, and the level was not completed; on ls20 L2 (two stops) no new version was accepted, and on cn04 L3 a re-certified plan ended without completing the level. We did not separate model inadequacy from verification and search cost.

6 Analysis using stop records

Success or failure alone does not identify a cause. The execution contract associates a routine's predictions with observed outcomes. Table 4 organises observed stop conditions, candidate causes, information needed to distinguish them, and related measurements. These records support diagnosis; they do not uniquely identify the cause of each failure.

Observation	Candidate causes	Diagnostic information	Measurement
No stored start or applicable family	Unseen game or level; family coverage	Coverage of routine sources	No certificate on 31 of 33 unseen games; one new level in 19.1 game-hours after replay.
Prediction mismatch at action k	Incorrect routine; hidden-state difference; changed rules	Same-start action sequence; changed cells	Two false certificates revoked after one action; all 16 planted deviations stopped at the specified point.
Same frame and action, different next frame	Internal state or history omitted from the frame	Level index; engine state; history-conditioned prediction	19.8% of 13,300 repeated keys; unchanged with the level index. Up to six actions separated 63% in sample; held-out accuracy 83.7% versus 74.4% on 649 answered cases (40% coverage).
Prediction failure after a level change	New effect; out-of-class effect; misapplication; unseen action	Effect representation; object identity	1,112 failures: 457, 445, 117, and 93 respectively.
Transitions predicted but no completion plan	Goal or object representation; unobserved transitions; search range	Goal-test coverage; reasons for skipped planning	Existing completion test held on 0 of 114 clear events. On NV25, 20 of 28 levels were skipped and eight planned, with no clears.
No corresponding action after a note	Message content; attention allocation	Response rate per note	Repair requests taken up 1/5; verification tools used in 2–3% of requests; advice followed 73–86%.

Table 4. Stop conditions, candidate causes, and diagnostic information.

Insufficient frame representation. Across 87 runs and 170,714 transitions, 2,635 of 13,300 repeated (game, frame, action) keys had multiple next frames. The rate was 20–49% on five games and near zero on three. Adding the level index did not change any count; the observed ambiguity occurred within levels.

Adding a history of up to six actions separated 63% of the ambiguous keys in sample. Only ten keys had at least two observations supporting every resulting branch. On held-out runs, the history-based predictor answered 649 cases, or 40% of eligible cases. Its accuracy was 83.7%, compared with 74.4% for frame-only majority prediction on those same cases. This establishes a predictive improvement where the method answers, not general recovery of hidden state.

Transfer between levels. An effect-template predictor without an LLM was fitted on levels 1 through k and evaluated on level k+1. Exact accuracy was 10.3% over 1,240 transitions in 37 level pairs. Of 1,112 failures, 445 involved effects outside the template class. This measures the limitations of the tested generalisation method, not the LLM's transfer ability.

7 Discussion

7.1 Retaining successful experience and reducing inference cost

The control agent can refer to successful histories but does not convert them into procedures for direct execution in a later run. It processes an average of approximately 75 k input tokens per request; with the solver disabled, it generates 0.36–0.51 M tokens per run on the solver-assigned levels. In the registered advice/authority comparison, those levels took approximately 80 game-minutes with advice and ten with direct execution. Section 5.2 gives the range including additional authority runs.

Actions repeating an already observed (frame, action) pair represented 62% of model actions on NV25 with the older harness and 1.4% on the public games with the current harness. Because both the environments and harnesses differ, the contrast cannot be attributed to a single factor. The distinction between interpreting recorded knowledge and executing a procedure has parallels with stages of skill acquisition (Fitts & Posner, 1967) and proceduralisation (Anderson, 1982), used here as architectural analogies.

The control configuration has no route for retaining newly acquired experience across runs in executable form. Frozen weights alone do not imply this limitation: Voyager retains code skills without weight updates. Updating weights is another option, but interference with previously acquired knowledge must be addressed (McCloskey & Cohen, 1989; Kirkpatrick et al., 2017).

Within a run, history is constrained by context length. The measured control configuration replaced its prompt with a summary 4.66 times per game-hour. Work on long-context use (Liu et al., 2024) is relevant, but these experiments do not isolate the causal effect of summary information loss on failure. Table 5 distinguishes the roles of weights, context, and routines.

Memory	Retention	Use and measured cost	Matching	Generalisation
Model weights	Across runs; fixed at test time	LLM inference	Not used for exact frame matching	Pretrained capability
Context	Within a run; 4.66 summary replacements per game-hour	Approximately 75 k input tokens per request; median 46 s to a turn-opening action	History, including summaries, interpreted by the LLM	In-context adaptation
Routines: replay and families	Stored procedures and implementations persist across runs	No generated LLM tokens during replay; median 0.24 s per replay action, including checks	Stored observations or family-specific conditions	Replay of the same procedure; a family's defined scope

Table 5. Three forms of memory in this system.

7.2 Relation to skill-acquisition efficiency

Chollet (2019) discusses intelligence in terms of skill-acquisition efficiency. We distinguish discovering a solution from using it repeatedly. On known tasks, replaying retained successes produced 46 completed levels with library v2, compared with a control mean of 34.5. Replay generated no LLM tokens, while the cost of exploring unsolved levels remained.

This result concerns reuse of the same tasks, not generalisation to new ones. Learning mechanic families with applicability conditions from successful trajectories remains open. Among levels completed by the model, the median ratio of human to agent actions was 1.00, 1.21, 1.08, 1.37, and 2.55 at L1–L5. Because this includes only completed levels, selection affects the comparison; it is not interpreted as a comparison of human and agent learning speed.

7.3 Application to physical AI

The contract stops a routine after an observed prediction mismatch. Even where a false certificate was revoked after one action, that action had already occurred. Neither harmless execution nor safe recovery by the LLM is guaranteed. A physical application would require a pre-action admissibility check and an independent controller for safe holding or retreat.

Exact frame hashes cannot be transferred directly to continuous values and noisy observations. Verification would need explicit tolerances and state estimation. Stochasticity also limits replay (Machado et al., 2018). The transferable design is the separation of applicability, monitored execution, and authority revocation. The measured 0.24 s and 46 s intervals belong to this game-execution system; they are not measurements of robotic control cycles.

7.4 Limitations

Local comparisons use eight public games with two to four runs per condition. Replay depends on known initial states and procedures. Family transfer was limited: two of 25 NV25 games received a certificate, and both certificates were revoked after one action. Kaggle evaluations have few repetitions and substantial same-kernel variation, so the contributions of individual components to hidden scores are not established.

The library comparison includes two harness fixes, and aggregate differences are not attributed to a single feature. The divergence experiment does not separate partial progress from the handover note. The history-based predictor improves accuracy on the 40% of eligible held-out cases it answers; it abstains on the rest. State repair therefore remains limited. The repair analysis of Section 5.7 re-reads two recorded runs; its two completed levels are levels the LLM alone also completes.

8 Conclusion

We implemented middleware that handles successful trajectory replay and certified game-mechanic models under a common execution contract. On public games, stored trajectories ran without LLM inference and matched all observations in 182 level replays. In the registered comparison (a1, a2 versus v1, v2), the same target levels were completed, and direct delegation to an executor used fewer LLM turns and generated tokens than presenting the same plans as advice. In the additional run a3, a level that was not certified was handled by the LLM, and counting that uncompleted level, the run’s assigned levels used more generated tokens than advice. Savings where a routine was applied and the cost of falling back when certification fails need to be evaluated separately.

All 16 planted prediction mismatches stopped replay at the specified action. Continuation from the reached state completed 11/16 levels, compared with 7/16 from the start, but the difference was not statistically significant and the note's individual effect was not established. Existing routines completed no levels on the 33 unseen games. With LLM-written models, every gate stop was followed by a new model version; three of

five stopped level attempts reached a version that agreed with every recorded transition, and two levels in one run were then completed by the harness in about as many actions as the LLM alone. This does not show performance beyond the controls.

The demonstrated contribution is verified reuse on known tasks and measurement of its scope. Further work will examine how to learn families that apply beyond a stored trajectory and how to select changes to the state representation after a prediction failure. One next step is to feed the diagnosis of Section 6 back into the repair of Section 5.7: when a stop record shows that the frame alone does not determine the next frame, the repair request should ask for a state that includes recent actions, which separated 63% of such cases in sample. Whether such repairs raise level completion as well as prediction is not tested here. The execution contract provides a basis for running such candidates under monitoring and stopping them after an observed mismatch.

Appendix A Experiment records and reproducibility

Experiment or analysis	Record (under docs/ unless specified)	Tool or output
Library (5.3, 5.4)	library_registration_2026-11-03	tools/library_harvest.py; research/solver_library/
Authority, advice, consistency (5.2, 5.1)	eca_advice_registration_2026-11-03	tools/advice_analysis.py; research/eca_advice/
Deviation (5.5)	library_divergence_registration_2026-11-03	tools/library_divergence.py
Action interval, context, unsolved levels (5.1, 5.6)	paper_outline_v4_2026-11-03	tools/paper_measures.py; research/paper_measures/
Unseen games (5.6)	nv25_solver_coverage_2026-11-03; unknown_dev_eca_alone_2026-10-30	tools/nv25_solver_coverage.py
Closure census (6)	closure_census_registration_2026-11-03	tools/closure_census.py; research/closure_census/
Within-run repetition (5.6, 7.1)	within_run_repeats_registration_2026-11-03	tools/within_run_repeats.py
Solver disabled (1, 5.1)	ecaoff_registration_2026-11-02	tools/level_tokens_ecaoff.py
Level-to-level transfer (6)	research/solver_library/transfer_v1.md	tools/transfer_measure.py
Repair after a stop (5.7)	refine_chain_registration_2026-11-05	tools/refine_chain_v2.py (supersedes refine_chain.py); research/refine_chain/
LLM-written models (5.7)	modeler_registration_2026-10-31; simplan_suppression_note_2026-10-29	tools/judge_modeler_colab.py; docs/modeler_colab_verdict_2026-10-31.txt

Linked submissions: 56836939 (ECAOD v1) and 57877794 (failure note). The ECA toolbox, harness patches, analysis code, and experiment records are released at github.com/kojima123/eca-paper-release (MIT-0; raw run outputs are excluded, see its NOTICE). Paths listed here identify their locations within that repository.

Appendix B An example of family execution and revocation

We describe reflection_orbit on ar25 in failure-note run f8. The family models a one-cell cursor movement followed by recomputation of positions reflected about an axis. P_ρ predicts the cursor's next cell and a reduction of the reflection plan by the executed action. It assumes that the axis and target footprint remain

observable. A mismatch in the cursor position, axis or target geometry, or recomputed plan suffix rejects the routine. Cursor, axis, and target parameters are obtained from the frame and observed transitions; a search under the model generates the plan.

In this run, the family executed 81 actions without the LLM and completed L1–L3. At L4, it verified 14 actions before the 15th, SPACE, produced a mismatch. The model expected a cycle operation, but six white 1×1 cells appeared at (31,19), (34,19), …, (46,19). The lease was revoked. The next LLM prompt contained the model, assumptions, falsification conditions, action, and observed changes, with a request to examine the missing behaviour while retaining the supported part. This is one stop record of the kind described in Section 6; it does not establish that the note was adopted or that repair succeeded.

Acknowledgements

This work uses Daniel Franzen's Milestone 2 notebook and Tufa Labs' Duck harness and includes evaluation on games published by NVIDIA. AI coding agents (Claude Code, Anthropic) assisted with experiments, analysis code, and manuscript preparation. ChatGPT was used for this language revision. The author is responsible for the research direction, experimental and submission decisions, and claims. Experimental values are based on the scripts and execution records listed in Appendix A.

References

1. Akyürek, E., Damani, M., Zweiger, A., et al. (2025). The surprising effectiveness of test-time training for few-shot learning. ICML 2025, PMLR 267, 942-963. arXiv:2411.07279.
2. Anderson, J. R. (1982). Acquisition of cognitive skill. Psychological Review, 89(4), 369-406.
3. ARC Prize Foundation (2026). ARC-AGI-3: A new challenge for frontier agentic intelligence. arXiv:2603.24621.
4. ARC Prize (2026). ARC Prize 2026. https://arcprize.org/competitions/2026 (accessed 2026-11-03).
5. Bessis, H., Cottaar, J., Pressman, I., Smit, D., Tešnar, M., Viel, S. (2026). Tufa Labs' winning solution for ARC-AGI-3 Milestone 1. Kaggle discussion 717133; notebook “Tufa Labs duck harness [June 30 milestone winner]”, kaggle.com/code/jeroencottaar; code github.com/Tufalabs/duck-harness.
6. Booch, G., Fabiano, F., Horesh, L., et al. (2021). Thinking fast and slow in AI. AAAI, 35(17), 15042-15046.
7. Chen, Q., Bellucci, A., Sun, Z., Jacucci, G. (2026). SkillDroid: Compile once, reuse forever. arXiv:2604.14872.
8. Cheng, J., Kumar, A., Lal, R., et al. (2025). WebATLAS: An LLM agent with experience-driven memory and action simulation. NeurIPS 2025 Workshop on Language Agents and World Models. arXiv:2510.22732.
9. Chollet, F. (2019). On the measure of intelligence. arXiv:1911.01547.
10. Daw, N. D., Niv, Y., Dayan, P. (2005). Uncertainty-based competition between prefrontal and dorsolateral striatal systems for behavioral control. Nature Neuroscience, 8(12), 1704-1711.
11. Fitts, P. M., Posner, M. I. (1967). Human Performance. Belmont, CA: Brooks/Cole.
12. Franzen, D. (2026). ARC-AGI-3 Milestone 2 Solution [Kaggle notebook]. kaggle.com/code/dfranzen/arc-agi-3-milestone-2-solution; code github.com/da-fr/arc-agi-3-solution.
13. Kirkpatrick, J., Pascanu, R., Rabinowitz, N., et al. (2017). Overcoming catastrophic forgetting in neural networks. PNAS, 114(13), 3521-3526.
14. Laird, J. E., Rosenbloom, P. S., Newell, A. (1986). Chunking in Soar: The anatomy of a general learning mechanism. Machine Learning, 1(1), 11-46.
15. Lehmann, J., Aioanei, A., Vahdati, S. (2026). Tycho: Active abstraction with programmatic world models for ARC-AGI-3. arXiv:2607.28287.
16. Lin, B. Y., Fu, Y., Yang, K., et al. (2023). SwiftSage: A generative agent with fast and slow thinking for complex interactive tasks. NeurIPS 36, 23813-23825.
17. Liu, N. F., Lin, K., Hewitt, J., et al. (2024). Lost in the middle: How language models use long contexts. TACL, 12, 157-173.
18. Liu, Z., Zhang, C., Zhao, X., et al. (2025). Beyond training: Enabling self-evolution of agents with MOBIMEM. arXiv:2512.15784.
19. Logan, G. D. (1988). Toward an instance theory of automatization. Psychological Review, 95(4), 492-527.
20. Machado, M. C., Bellemare, M. G., Talvitie, E., Veness, J., Hausknecht, M., Bowling, M. (2018). Revisiting the Arcade Learning Environment: Evaluation protocols and open problems for general agents. JAIR, 61, 523-562.

21. McCloskey, M., Cohen, N. J. (1989). Catastrophic interference in connectionist networks: The sequential learning problem. Psychology of Learning and Motivation, 24, 109-165.
22. Norman, D. A., Shallice, T. (1986). Attention to action: Willed and automatic control of behavior. In Davidson, Schwartz, Shapiro (Eds.), Consciousness and Self-Regulation, vol. 4, 1-18. New York: Plenum.
23. Pourcel, J., Colas, C., Oudeyer, P.-Y. (2025). Self-improving language models for evolutionary program synthesis: A case study on ARC-AGI. ICML 2025, PMLR 267, 49659-49688.
24. Rodionov, S. (2026). Executable world models for ARC-AGI-3 in the era of coding agents. AGI 2026, LNCS, 198-210. arXiv:2605.05138.
25. Schmidt, R. A. (1975). A schema theory of discrete motor skill learning. Psychological Review, 82(4), 225-260.
26. Sha, L. (2001). Using simplicity to control complexity. IEEE Software, 18(4), 20-28.
27. Sestak, F., Aichberger, L., Ielanskyi, M., Klambauer, G. (2026). The last dance: short overview of a 7th-place ARC-AGI-3 entry. ARC Prize 2026 Kaggle discussion 756428, 2026-11-06.
28. Skoutnev, A., Acharya, K., Longhitano, G., Udell, M., Ellis, K., Drori, I. (2026). Twin: Playing an unknown game with a test-time digital twin. arXiv:2608.14490.
29. Sun, H., Zhuang, Y., Kong, L., Dai, B., Zhang, C. (2023). AdaPlanner: Adaptive planning from feedback with language models. NeurIPS 36, 58202-58245.
30. Vaandrager, F., Garhewal, B., Rot, J., Wißmann, T. (2022). A new approach for active automata learning based on apartness. TACAS 2022, LNCS 13243, 223-243.
31. Wang, G., Xie, Y., Jiang, Y., et al. (2024). Voyager: An open-ended embodied agent with large language models. TMLR. arXiv:2305.16291.
32. Wolpert, D. M., Ghahramani, Z., Jordan, M. I. (1995). An internal model for sensorimotor integration. Science, 269(5232), 1880-1882.
33. Yao, S., Shinn, N., Razavi, P., Narasimhan, K. (2025). Tau-bench (tau-bench): A benchmark for tool-agent-user interaction in real-world domains. ICLR 2025. arXiv:2406.12045.
34. Zeng, G., Wang, J., Ma, W., et al. (2026). Schema: Discovering unknown environments via agentic program induction. arXiv:2609.39140; project report schema-harness.github.io (2026-07-15).